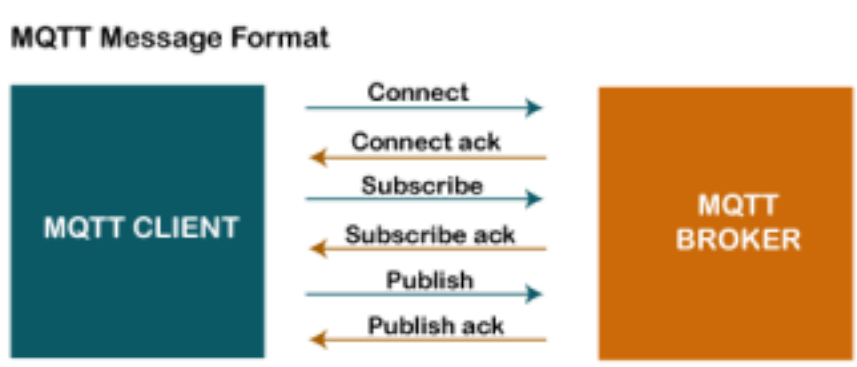


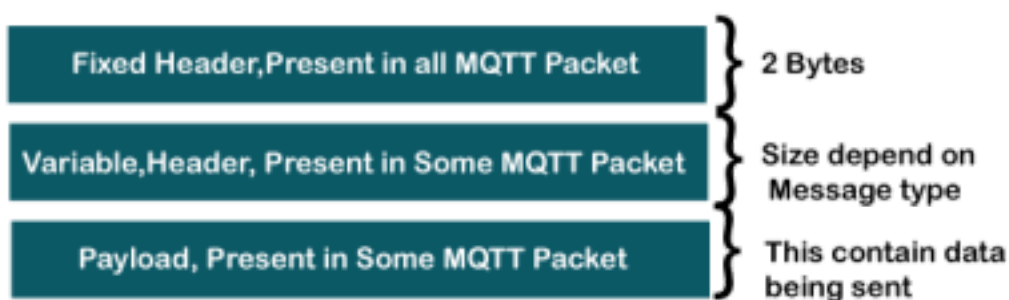
Suppose a device has a temperature sensor and wants to send the rating to the server or the broker. If the phone or desktop application wishes to receive this temperature value on the other side, then there will be two things that happened. The publisher first defines the topic; for example, the temperature then publishes the message, i.e., the temperature's value. After publishing the message, the phone or the desktop application on the other side will subscribe to the topic, i.e., temperature and then receive the published message, i.e., the value of the temperature. The server or the broker's role is to deliver the published message to the phone or the desktop application.

MQTT Message Format



The MQTT uses the command and the command acknowledgment format, which means that each command has an associated acknowledgment. As shown in the above figure that the connect command has connect acknowledgment, subscribe command has subscribe acknowledgment, and publish command has publish acknowledgment. This mechanism is similar to the handshaking mechanism as in TCP protocol.

MQTT Packet Structure



The MQTT message format consists of 2 bytes fixed header, which is present in all the MQTT packets. The second field is a variable header, which is not always present. The third field is a payload, which is also not always present. The payload field basically contains the data which is being sent. We might think that the payload is a compulsory field, but it does not happen. Some commands do not use the payload field, for example, disconnect message.

Fixed Header

Let's observe the format of the fixed header.

Fixed Header

BIT	7	6	5	4	3	2	1	0
Byte1	MQTT Control Packet Type				Flag specific to each MQTT Packet type			
Byte2...	Remaining Length							

As we can observe in the above format that the fixed header contains two bytes. The first byte contains the following fields:

- **MQTT Control Packet Type:** It occupies 4 bits, i.e., 7 to 4-bit positions. This 4-bit is an assigned value, and each bit represents the MQTT control packet type.
- **Flag specific to each MQTT packet type:** The remaining 4-bits represent flag specific to each MQTT packet type.

The byte 2 contains the remaining length, which is a variable-length byte integer. It represents the number of bytes remaining in a current control packet, including data in the variable header and payload. Therefore, we can say that the remaining length is equal to the sum of the data in the variable header and the payload.

MQTT Control Packet Types

MQTT Control Packet Types

Name	Value	Direction of flow	Description
Reserved	0	Forbidden	Reserved
CONNECT	1	Client to Server	Connection request
CONNACK	2	Server to Client	Connect acknowledgment
PUBLISH	3	Client to Server or Server to Client	Publish message
PUBACK	4	Client to Server or Server to Client	Publish acknowledgment(QoS1)
PUBREC	5	Client to Server or Server to Client	Publish received(QoS2 delivery part 1)
PUBREL	6	Client to Server or Server to Client	Publish release(QoS 2 delivery part 2)
PUBCOMP	7	Client to Server or Server to Client	Publish complete (QoS 2 delivery part 3)
SUBSCRIBE	8	Client to Server	Subscribe request
SUBACK	9	Server to Client	Subscribe acknowledgment
UNSUBSCRIBE	10	Client to Server	Unsubscribe request
UNSUBACK	11	Server to Client	Unsubscribe acknowledgment
PINGREQ	12	Client to Server	PING request
PINGRESP	13	Server to Client	PING response
DISCONNECT	14	Client to Server or Server to Client	Disconnect notification
AUTH	15	Client to Server or Server to Client	Authentication exchange

The above table shows the control packet types with 4-bit value and direction flow. As we can observe that every command is followed by acknowledgment like CONNECT has CONNACK, PUBLISH has PUBACK, PUBREC, PUBREL, and PUBCOMP, SUBSCRIBE has SUBACK, UNSUBSCRIBE has UNSUBACK.

Flag Bit

Flags Bit

MQTT Control Packet	Fixed Header Flags	Bit 3	Bit 2	Bit 1	Bit 0
CONNECT	Reservd	0	0	0	0
CONNACK	Reservd	0	0	0	0
PUBLISH	Used in MQTTv5.0	DUP	QoS		RETAIN
PUBACK	Reservd	0	0	0	0
PUBREC	Reservd	0	0	0	0
PUBREL	Reservd	0	0	0	0
PUBCOMP	Reservd	0	0	0	0
SUBSCRIBE	Reservd	0	0	0	0
SUBACK	Reservd	0	0	0	0
UNSUBSCRIBE	Reservd	0	0	0	0
UNSUBACK	Reservd	0	0	0	0
PINGREQ	Reservd	0	0	0	0
PINGRESP	Reservd	0	0	0	0
DISCONNECT	Reservd	0	0	0	0
AUTH	Reservd	0	0	0	0

OMIARUP5D19 SP1 18

The above table shows the flag value associated with each command. Here, reserved refers to future use, which means that it is not being used right now. In the case of PUBLISH command, flag bits are further divided into DUP, QoS, and RETAIN, where DUP is a duplicate delivery of a PUBLISH packet, QoS is Quality of Service, and RETAIN is retained message flag.

Remaining length

The remaining length is a variable-length integer that denotes the number of bytes remaining within the current control packet, including data in the variable header and the payload. Therefore, the remaining length is equal to the data in the variable header plus payload.

$$\text{Remaining length} = \text{length of variable header} + \text{length of payload}$$

For example, if the length of the variable header is 20 and the length of the payload is 30, then the remaining length is 50.

The remaining length can be used upto 4 bytes, and it starts from 2 bytes and can be used upto 4 bytes.

This field uses 7-bit for the lengths, and MSB bit can be used to continue a flag. If the continuation flag is 1, the next byte is also a part of the

remaining length. If the continuation flag is 0, a byte is the last one of the remaining length. **Variable**

header

Some types of MQTT control packet types contain an optional field also, i.e., variable header component.

This field resides between the fixed header and the payload. The content of the variable header depends upon the packet type. The variable header contains the packet identifier field, which is common in several packet types. The variable header component of many MQTT control packet types includes 2-byte integer, i.e., the packet identifier field.

The given list below contains the packet identifier field:

- PUBLISH
- PUBACK
- PUBREC
- PUBREL
- PUBCOMP
- SUBSCRIBE
- SUBACK
- UNSUBSCRIBE
- UNSUBACK

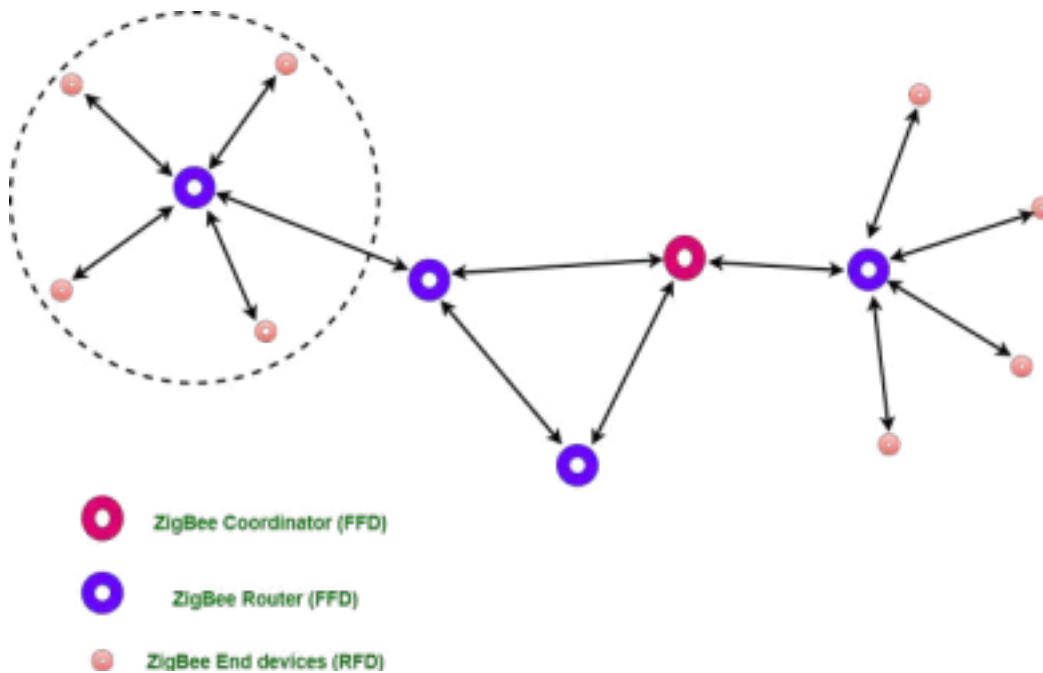
➤ ZIGBEE

ZigBee is a Personal Area Network task group with low rate task group 4. It is a technology of home networking. ZigBee is a technological standard created for controlling and sensor the network. As we know that ZigBee is the Personal Area network of task group 4 so it is based on IEEE 802.15.4 and is created by Zigbee Alliance.

ZigBee is a standard that addresses the need of very low-cost implementation of Low power devices with Low data rate for short-range wireless communications.

Types of ZigBee Devices:

- **Zigbee Coordinator Device** – It communicates with routers. This device is used for connecting the devices.
- **Zigbee Router** – It is used for passing the data between devices.
- **Zigbee End Device** – It is the device that is going to be controlled



General

Characteristics of Zigbee Standard:

- Low Power Consumption
- Low Data Rate (20- 250 kbps)
- Short-Range (75-100 meters)
- Network Join Time (~ 30 msec)
- Support Small and Large Networks (up to 65000 devices (Theory); 240 devices (Practically))
- Low Cost of Products and Cheap Implementation (Open Source Protocol)

Operating Frequency Bands (Only one channel will be selected for use in a network):

1. **Channel 0:** 868 MHz (Europe)
2. **Channel 1-10:** 915 MHz (US and Australia)
3. **Channel 11-26:** 2.4 GHz (Across the World)

Zigbee Network Topologies:

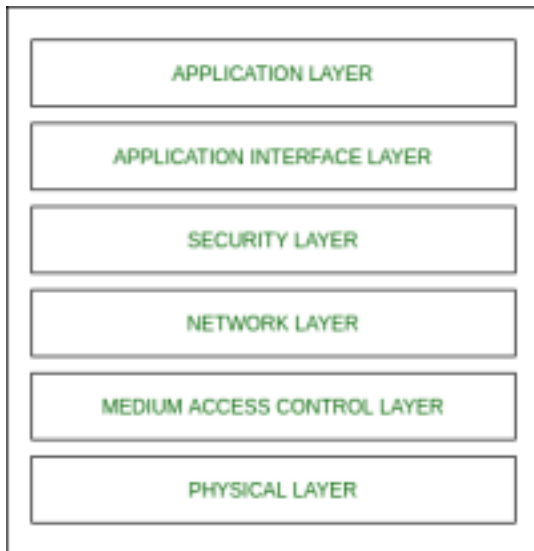
- Star Topology (ZigBee Smart Energy)
- Mesh Topology (Self Healing Process)
- Tree Topology

Architecture of Zigbee:

Zigbee architecture is a combination of 6 layers.

1. Application Layer
2. Application Interface Layer

3. Security Layer
4. Network Layer
5. Medium Access Control Layer
6. Physical Layer



- The Application layer is present at the user level.
- The Application Interface Layer, Security Layer, and Network Layer are the Zigbee Alliance and they are used to store data and they use the stack.
- Medium Access control and the Physical layer are the IEEE 802.15.4 and they are hardware which are silicon means they accept only 0 and 1.

Channel Access:

1. **Contention Based Method** (Carrier-Sense Multiple Access With Collision Avoidance Mechanism)
2. **Contention Free Method** (Coordinator dedicates specific time slot to each device (Guaranteed Time Slot (GTS)))

Applications of ZigBee Technology

Its three major USPs are low-cost, low-power consumption and having faster wireless connectivity, the ZigBee protocol caters to a lot of applications like industrial automation, home automation, smart metering, smart grids etc. Also with its low-power requirements, it ensures seamless operation of various sensor equipments offering years of battery-life. Here are some of the areas where ZigBee is widely used.

- **Industrial Automation:** ZigBee offers a faster and low-cost communication that can communicate with almost all devices in factories and centralise them at one place making it easy for you to monitor every process and thereby optimise the control process. ZigBee protocol also finds its presence in many medical and scientific equipments such as personal chronic monitoring, sports and fitness trackers, and can even be used for remote patient monitoring.

- **Smart Metering and Smart Grid Monitoring:** In case of smart metering, ZigBee is used for better energy consumption response, security over power theft, pricing support etc. Additionally in case of smart grids, ZigBee is even used for reactive power management, fault locations, remote temperature monitoring, etc.

- **Home Automation:** ZigBee is one of the most widely used protocol in most of the home automation equipments. Right from offering lighting system solutions, sensor responsive solutions to security solutions and surveillance, ZigBee has its presence everywhere.

ZigBee for Home Automation



ZigBee protocol is widely used for home automation solutions and caters to complete holistic solutions of lighting control, security control, comfort control and even energy management.

There are several well-known global brands in home automation that use ZigBee for their devices. Since ZigBee is cross-compatible and interoperable, it makes managing multi vendor devices easy and simple. If a device is ZigBee Home Automation (HA 2.1) compliant, you can be rest assured that it will work with your automation system, irrespective of the vendor.

The mesh-routing network of ZigBee wherein one device can talk to multiple device and data packets travel on no fixed routes, offers better flexibility and faster communication across devices.

Some of the features of ZigBee for Home Automation include:

- Simplified setup and maintenance
- Ideal for new construction and remodelling

- ZigBee gives access to devices anywhere from the world just from your smartphone
- Monitors power use and allows you to turn on/off devices from remote locations
- Built in security with interference avoidance techniques ensures better/enhanced security and worry-free operations.
- Help you customise lighting scenes based on daily schedules, events and activities.
- Due to low-power consumption of the ZigBee protocol, your security sensors can work for a period of 7 years.

➤ Bluetooth

Bluetooth has been in the tech market as a wireless channel of connection between devices since Ericsson invented it in 1994. Since then, Bluetooth technology has evolved and has become the go-to wireless connectivity solution for wearables, gadgets, and other devices. Nowadays, you will find Bluetooth everywhere; cars, speakers, wearables, medical devices, wireless headphones, shoes, etc. If you own any modern device, it is safe to assume that you have encountered and used Bluetooth technology at one point or the other. In other words, Bluetooth is a short-range wireless technology medium used for exchanging data between two electronic devices (usually mobile) over a short distance. This process completely eliminates the primitive use of cables for connectivity. A typical example is how you can listen to music with a headset on the go without having to plug it into the headset jack of your mobile device.



Bluetooth exchange works using UHF radio waves, otherwise known as short wave radio, with radio bands ranging from 2.402 GHz to 2.480 GHz and building a Personal Area Network (PAN). Typically, a master Bluetooth device can connect to a maximum of seven devices at a go. Still, some Bluetooth devices do not have the capacity to connect up to this number of devices. However, this kind of connection is called a piconet, an ad hoc computer network created at that moment using Bluetooth technology. And in this technology system, connected operate in a master-slave relationship. For example,

suppose you initiate a connection between a phone and a wireless headset through a headset; in that case, the headset becomes the master (the initiator), and the phone is the slave. Subsequently, both devices can

switch roles and have the phone operate as the master, while the headset becomes the slave. Ultimately, in a Bluetooth piconet, it is possible for a master to have seven slaves; and for a slave to have more than one master.

Bluetooth Classic and Bluetooth Low Energy (BLE)

There are two Bluetooth variants of the Bluetooth technology; hence all Bluetooth devices can be classified into two categories – Bluetooth Classic and Bluetooth Low Energy (BLE). On the one hand, the Bluetooth Classic is usually used in wireless speakers, headsets, and car infotainment systems. On the other hand, Bluetooth Low Energy (just as the name implies) is more prominent in applications that are keen on power consumption and transfer small amounts of data less often. In other words, BLE is commonly found in battery-powered devices like mobile phones, sensor devices, etc. As opposed to the Bluetooth Classic that consumes high energy, Bluetooth Low Energy thrives on reduced power consumption and cost, even while maintaining a similar communication range as Bluetooth Classic.

It is important to note that these two kinds of Bluetooth devices are inharmonious even when they share the same brand and specification document. That is to say, a Bluetooth Classic cannot work together with Bluetooth Low Energy. So, it is not farfetched why some devices like smartphones integrate both Bluetooth variants to communicate with and connect to either type of Bluetooth present in other devices.

	Classic Bluetooth technology	Bluetooth low energy technology
Data payload throughput (net)	2 Mbps	~100 kbps
Robustness	Strong	Strong
Range	Up to 1000m	Up to 250m
Local system density	Strong	Strong
Large scale network	Weak	Good
Low latency	Strong	Strong
Connection set-up speed	Weak	Strong
Power consumption	Good	Very strong
Cost	Good	Strong

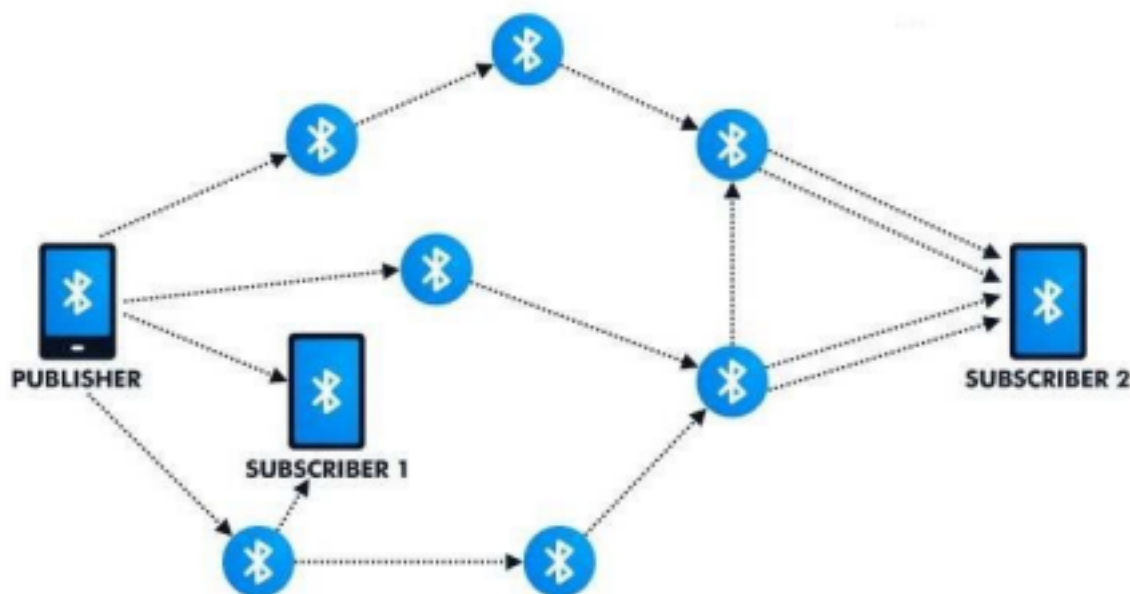
Bluetooth In IoT

Despite its common uses, Bluetooth has become a household name in the Internet of Things community. It is a serious technology used for IoT applications. Apart from being the ubiquitous solution for hands-free calling and wireless transmission technology for

audio, Bluetooth technology is leading in consumer and business IoT. Since a device-to device connection is expected to be fast, seamless, and wireless in the Internet of Things, internet of things Bluetooth (Bluetooth IoT) is highly deployed because of its no internet function on the one hand and its capability to create large-scale device networks via Bluetooth mesh, on the other hand.

Bluetooth Mesh Networking

Bluetooth mesh IoT is a computer mesh networking system based on Bluetooth Low Energy (BLE) that allows for many-to-many communication among connected devices over a Bluetooth radio. In a Bluetooth mesh IoT network, every message has a source and destination address through which devices publish messages to their destinations, which is a single thing, group of things, or everything.



Bluetooth mesh IoT networking is a game changer for wireless device networks. It is no surprise that it sets a stage for a new wave of connectivity from whole-building networks to city-wide smart services, especially in the present era of the home, building, community, and industrial automation.

Why Bluetooth in IoT?

Bluetooth Low Energy in IoT can help IoT devices conserve energy by keeping the devices in sleep mode when they are not in use, then let users exit the mode when connected or reconnected. Bluetooth Low Energy in IoT is ideal for IoT applications because, contrary to the classic Bluetooth applications, which reconnect to devices at a time of six seconds or more, IoT BLE applications can quickly pair and reconnect with devices in six milliseconds instead.

In IoT BLE, a device can function in three stages; the Advertising stage, Scanning stage, and Connected stage. In a scenario where you want to integrate two BLE devices with each other, one device has to advertise. In contrast, the other has to scan for the device

advertising before subsequently initiating a connection. Advertising basically involves broadcasting packets that allow another scanning device to find them.

Bluetooth IoT Devices

Advertising is deployed in all Bluetooth IoT devices, but one prominent application that exclusively functions in this state is the Beacon technology. Beacon devices, like the MOKOBlue M1 Ultra-thin Beacon, stay in the Advertising mode while broadcasting data to other devices that they can explore and read such data from. Since advertising data capacity is increased in Bluetooth 5.0, Beacons can unlock new IoT applications and use cases by transmitting more data.



Properties of Bluetooth network

- **Standard:** Bluetooth 4.2
- **Frequency:** 2.4GHz
- **Range:** 50-150m
- **Data transfer rates:** 3Mbps

Advantages of Bluetooth network

- It is wireless.
- It is cheap.
- It is easy to install.
- It is free to use if the device is installed with it.

Disadvantages of Bluetooth network

- It is a short-range communication network.

- It connects only two devices at a time.

COMARUP20216 591 08

➤ Constrained Application Protocol

CoAP is a customary client-server IoT protocol. It enables clients to make requests for web transfers as per the need of the hour. On the other hand, it also lets supporting servers to respond to arriving requests. In summary, devices' nodes in the IoT ecosystem are enabled to interact over through CoAP only.

CoAP and HTTP follow the same working procedure. However, CoAP attains its functionality via asynchronous transactions (using UDP). It utilizes the POST, GET, PUT, and DELETE calls. That's the reason why API security is of higher grade while CoAP is active as it is an RPK and PSK-certified protocol.

CoAP is compatible with 4 types of information exchange:

1. Acknowledgments confirm the completion or failure of an event.
2. Confirmable are the messages that are resent on time out until the confirmation of successful sending doesn't arrive.
3. Reset messages are empty, with confirmable as their nature.
4. Non-confirmable information is just sent and has no guarantee of successful delivery. There is no acknowledgment of success either.

Key traits of CoAP are:

- Works for devices in the same network types.
- Enables data transmission, to and fro, for the general internet-enabled nodes and network-connected devices.
- Works really fine for SMSs shared over mobile network connectivity.
- Suitable for internet-operative applications that use connected devices/sensors and have resource limitations.
- Capable of translating HTTP, supports multicast, and exerts the bare minimum cost burden.
- Only helps machines to communicate (in the network).

CoAP Architecture

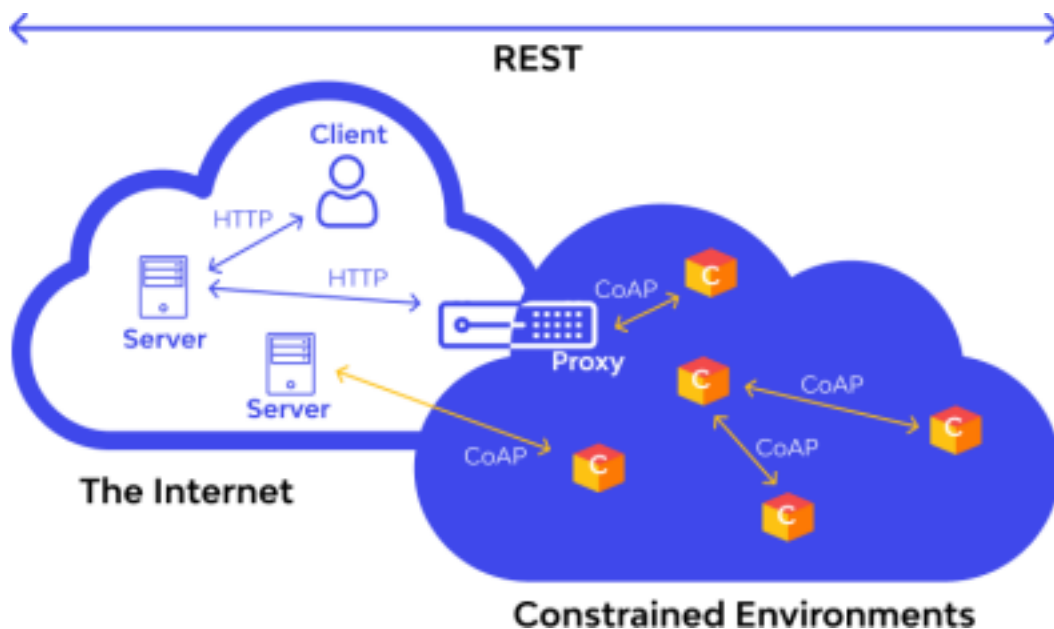
The WWW and the constraints ecosystem are the 2 foundational elements of the CoAP protocol architecture. Here, the server monitors and helps in communication happening using CoAP and HTTP while proxy devices bridge the existing gap for these 2 ecosystem, making the communication smoother.

CoAP allows HTTP clients (also called CoAP clients here) to talk or exchange data/information with each

other within resource constraints. While one tries to understand this architecture, gaining acquaintances with some key terms is crucial:

- Endpoints are the nodes that host have knowledge of;

- Client sends requests and replies to incoming requests;
- Server gets and forwards requests. It also gets and forwards the messages received in response to the requests it had processed.
- Sender creates and sends the original message.
- Recipient gets the information sent by the client or forwarded by the server.



CoAP Function

The key role of CoAP is to act like HTTP wherever restricted devices are a part of communication. While filling the gap of HTTP, it enables devices like actuators and sensors to interact over the internet.

The devices, involved in the process, are administered and controlled by considering data as a system's component. CoAP protocol can operate its functions in an environment having reduced bandwidth and extreme congestion as it consumes reduced power and network bandwidth.

Networks featuring intense congestion and constrained connectivity are not ideal conditions for TCP-based protocols to carry out their responsibilities. CoAP comes as a rescuer at this place and

supports the web transfers.

Web transfers happening using satellites and covering long distances can be accomplished with full perfection using CoAP. Networks featuring billions of nodes take the help of the CoAP protocol for information exchange.

COMARUP20218 591 08

Regardless of the function handled or role played, CoAP promised security of highest grade as DTLS parameters as default security parameter; the counterpart of 128 bit RSA keys.

Speaking of its deployment, it's simple and hassle-free. It can be implemented from scratch for a straightforward application.

For the application ecosystem where CoAP is not desirable, generic implementations are offered for various platforms. Most of the CoAP implementations are done privately while few are published in open-source libraries like MIT license.

CoAP Features

The defining features that place CoAP protocol separate from other protocols are as stated next. As it shares great similarities with HTTP, developers face bare minimum difficulties while using it.

CoAP is an integration-friendly protocol and can be paired easily with applications using cross-protocol proxies. Seamlessly, it integrates with JSON, XML, CBOR, and various other data formats. In the process, the web client doesn't get hints about a sensor resource being accessed.

Developers are endowed with various payloads and have the freedom to make a choice to bring the ideal payload into action.

The successful IoT device/application demands the usage of billions of nodes at a time. CoAP is designed to handle such huge mode amounts with full perfection while keeping the overheads under control. It can operate on tons of microcontrollers while using the least possible resources. RAM space as low as 10KiB and code space as 100 KiB is enough for CoAP.

As resources demanded by CoAP are on the minimum side, it keeps the wastes under control. There is no need to deploy a hefty transport stack for web transfers. The header and encoding, used for message processing, are compact and don't cause any fragments on the link layer. At a time, it supports the functions of multiple servers.

CoAP offers a comprehensive resource directory to spot the properties of the node.

CoAP is verified by RFC 7252, is developed for the future, and is able to deal with congestion control issues.

ICM&ARUPSD2016 SP1 08

CoAP Layer

The protocol works through its two layers:

1. CoAP Messages Model

It makes UDP transactions possible at endpoints in the confirmable (CON) or non confirmable (NON) format. Every CoAP message features a distinct ID to keep the possibilities of message duplications at bay.

The 3 key parts involved to build this layer are binary header, computer option, and payload.

As explained before, confirmable texts are reliable and easy-to-construct message that are fast and are resent until the receipt of a confirmation of successful delivery (ACK) with message ID.

2. CoAp Request/Response Model

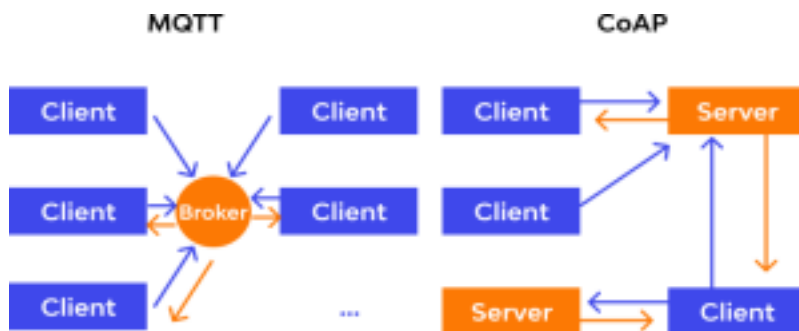
This layer takes care of CON and NON message requests. Acceptance of these requests depend on server's availability. Cases are:

1. If idle, the server will handle the request right away. If a CON, the client will get an ACK for it. If the ACK is shared as a Token and differs from the ID, it is essential to map it properly by matching request-response pairs.
2. If there is a delay or wait involved, the ACK is sent but as an empty text. When its turn arrives, the request is processed and the client gets a fresh CON.

The key traits of the Request/Response model are mentioned next:

- Request or response codes for CoAP are same as for the HTTP, except for the fact that they are in the binary format (0-8 byte Tokens) in CoAP's case.
- Request methods for making calls (GET, PUT, POST, and DELETE) are declared in the process.
- A CON response could either be stored in an ACK message or forward as CON/NON.

ICM&ARUPSD2016 SP1 08



CoAP vs MQTT

As there are great similarities, we won't blame you if you consider these two identical. For instance, they both are used for IoT devices as they both necessitate less amounts of network packets causing more power-optimized performance, less storage consumption, and longer battery power.

CoAP vs MQTT

MQTT	CoAP
This model has publishers and subscribers as main participants	Uses requests and responses
Central broker handles message dispatching, following the optimal publisher to client path.	Message dispatching happens on a unicasting basis (one-to-one). The process is same as HTTP.
Event-oriented operations	Viable for state transfer
Establishing a continual and long-lasting TCP connection with the broker is essential for the client.	Involved parties use UDP packets (async) for message passing and communication.
No message labeling but have to use diverse messages for different purposes.	It defines messages properly and makes its discovery easy.

➤ UDP

The TCP/IP protocol despite being most common protocol stack on internet is not much suitable for IoT applications due to large overhead. It is more suitable for applications where reliable delivery of data with high bandwidth in hand is required. The IoT applications generally have limited bandwidth and need swift transfer of small data packets. In such case, the UDP/IP stack is far better than TCP/IP.

The User Datagram Protocol (UDP) is the simplest transportation layer protocol used primarily for establishing low-latency and loss tolerating connections between applications on the communication network. Both TCP and UDP run on the top of Internet Protocol (IP) that is why they are referred as TCP/IP and UDP/IP.

UDP is a connectionless protocol which means the sender just transmits the data without waiting for the connection with the receiver. It is an unreliable protocol when compared with TCP. There is no error checking mechanism or correcting mechanism involved in data transmission which results in using less bandwidth. UDP protocol just sends the packets (or datagram). There is no acknowledgement guarantee of packet received by the other end. It allows for less data overhead and delays.

To achieve higher performance, the protocol allows individual packets to be dropped (with no retries) and UDP packets to be received in a different order than they were sent, as dictated by the application.

Features of UDP –

The UDP protocol stack has the following features –

UDP can be used when acknowledgement of data does not hold any significance. 2) It is great for data flowing in one direction.

3) It is connectionless protocol.

4) It does not provide any congestion control mechanism.

5) It is a suitable protocol for streaming applications such as video conference applications, computer games etc.

UDP Datagrams –

UDP traffic works through packets called datagram, with every datagram consisting of a single message unit. The header details are stored in the first eight bytes, but the rest is what holds onto the actual message. The UDP datagram header can be divided into four parts each of which is two bytes long. These

parts are as follow –

1) **Source Port** – This 16 bits (2 bytes) information is used to identify the sender port which will send the data. A valid UDP port number ranges from 0 to 65535.

COMARUPD2016 SP1 08

2) **Destination Port** – This 16 bits information is used to identify the receiver's port on which the data will be received. A valid UDP port number ranges from 0 to 65535. This field identifies the receiver's port and is required.

3) **Length** – The length field specifies the entire length of the UDP packet (UDP header and UDP data). This individual field is 16-bits field. The minimum length of the Length field is 8 bytes in case of no UDP data.

4) **Checksum** – This field stores the checksum value generated by the sender before sending the data to the receiver. UDP checksums protect message data from being corrupted. The checksum value represents an encoding of the datagram data calculated first by the sender and later by the receiver. In UDP, checksum is optional, as opposed to TCP where checksum is mandatory.

Advantages of UDP –

The UDP/IP has the following advantages over TCP/IP stack –

1) It is better than TCP for applications that require constant data flow, bulk data and which require more swiftness than reliability.

2) For multicast and broadcast purposes, UDP is best suited because it supports point to multipoint transmission method. The sender does not need to keep track of retransmission of data for multiple receivers in contrast with the TCP/IP where sender needs to take care of each packet.

3) There is small packet header overhead in UDP (only 8 bytes) whereas TCP has 20 bytes of header.